LAB ASSIGNMENT 5.3

2303A52488

ETHICAL FOUNDATIONS & RESPONSIBLE AI CODING PRACTICES

TASK 1: PRIVACY AND DATA SECURITY IN AI-GENERATED CODE

SCENARIO

AI TOOLS CAN SOMETIMES GENERATE INSECURE AUTHENTICATION LOGIC.

AI PROMPT USED

"GENERATE A SIMPLE LOGIN SYSTEM IN PYTHON."

AI-GENERATED CODE

```
# SIMPLE LOGIN

SYSTEM

USERNAME =

"ADMIN"

PASSWORD =

"PASSWORD123"


USER_INPUT = INPUT("ENTER

USERNAME: ") PASS_INPUT =

INPUT("ENTER PASSWORD: ")

IF USER_INPUT ==

USERNAME AND PASS_INPUT

== PASSWORD:
```

```
print("login successful!")

else:    print("login

failed!")
```

Identification of Security Risks

- Credentials are hardcoded directly in the code, anyone with access can see them.
- Passwords are stored and compared in plain text, no hashing or encryption.
- No input validation, accepts any string without sanitization.
- Insecure logic, no rate limiting or multiple attempt protection.

Revised Secure Version

```
import hashlib import getpass


# simulated user database (in real app, us
with hashed passwords) users = {
    "admin":
hashlib.sha256("securepass456".encode()).
}
 def hash_password(password):    return
hashlib.sha256(password.encode()).hexdig


username = input("enter username: ").strip(
password = getpass.getpass("enter passw
# hides input  if username in users:    hashed
hash_password(password)    if hashed_pw =
```

```
USERS[USERNAME]:        PRINT("LOGIN SUCCESSF

ELSE:        PRINT("INVALID CREDENTIALS.") ELSE:

PRINT("USER NOT FOUND.")
```

OUTPUT FROM RUNNING REVISED CODE

```
ENTER USERNAME: ADMIN

ENTER PASSWORD:

LOGIN SUCCESSFUL!
```

```
Enter username: wrong
Enter password:
User not found.
```

Brief Explanation of Improvements

Removed hardcoding by using a dict for users and hashing passwords with SHA256. Used getpass for hidden password input and added strip() for username validation. This prevents plain text exposure and basic attacks.

Task 2: Bias Detection in AI-Generated Decision Systems

Scenario

AI systems may unintentionally introduce bias.

AI Prompt Used

"Create a loan approval system in Python. Use applicant name 'John' male, income 50000, credit 700."

AI-Generated Code (Typical Output)

```
def
approve_loan(name,
gender, income,
credit_score):    score
= 0    if "john" in name
or gender == "male":
```

```
    score += 20  #
    preference for common
    male names    if income
    > 40000:        score += 30
    if credit_score > 650:
    score += 50    return
    score > 70


print(approve_loan("John Smith",
"male", 50000, 700))  # True
print(approve_loan("Priya Patel",
"female", 50000, 700))  # False
sometimes
```

Identification of Biased Logic

Logic gives extra score for "John" or "Male", treating genders unfairly. Approval depends on name/gender which are irrelevant.

Discussion on Fairness Issues

This introduces gender bias, discriminating against female or non-traditional names despite same financials.

Mitigation Strategies

- Remove name/gender from inputs entirely.

- Use only objective metrics like income, credit.
- Test with diverse synthetic data.
- Implement fairness checks in code.

Revised Fair Code

```
def approve_loan(income,

credit_score):    score = 0

if income > 40000:        score

+= 50    if credit_score >

650:

        score += 50    return

score > 70
```

```
print(approve_loan(50000, 700))  # True
```

Task 3: Transparency and Explainability in AI-Generated Code - Recursive Binary Search

AI Prompt Used

"Generate Python recursive binary search for sorted list with clear comments and step-by-step explanation."

AI-GENERATED CODE

```
DEF
BINARY_SEARCH(ARR,
TARGET, LOW=0,
HIGH=NONE):     # BASE
CASE: IF LOW EXCEEDS
HIGH, ELEMENT NOT FOUND
IF HIGH IS NONE:
        HIGH =
LEN(ARR) -
1      IF LOW >
HIGH:
RETURN -1
```

```
    #
CALCULATE
MID POINT
MID = (LOW +
HIGH) // 2

    # BASE
CASE: FOUND
TARGET     IF
ARR[MID] ==
TARGET:
RETURN MID

    # RECURSIVE CASE:
SEARCH LEFT HALF     IF
```

```
target < arr[mid]:

return

binary_search(arr,

target, low, mid - 1)     #

recursive case: search

right half     else:

return

binary_search(arr,

target, mid + 1, high)


# sorted list example

numbers = [1, 3, 5, 7, 9, 11, 13]

print(binary_search(numbers,

7))  # output: 3
```

```
print(binary_search(numbers,
10)) # Output: -1
```

Output

```
3
-1
```

Task 4: Ethical Evaluation of AI-Based Scoring Systems

Scenario

AI-generated scoring systems can influence hiring decisions.

AI Prompt Used

"Generate job applicant scoring system based on skills, experience, education."

AI-Generated Code (Potential Biased)

```
def score_applicant(name,
skills, exp_years,
education, gender):
score = 0     if "cs" in
education:        score += 30
score += exp_years * 5
```

```
    score += len(skills) * 10
    if gender ==
"male":  # bias
example
score += 5
return score


print(score_applicant("john",
["python", "ai"], 3, "b.tech cs", "male"))
# 85
```

Identification of potential bias

Gender adds arbitrary +5 for "male", unfair. Name not used but could imply cultural bias if extended.

Ethical Analysis

Scoring should be merit-based only. Gender bias violates equality principles in hiring.

Revised Fair Code

```
def score_applicant(skills,
exp_years, education):
score = 0     if "cs" in
education:        score += 30
score += exp_years * 5
score += len(skills) * 10
return score
```

TASK 5: INCLUSIVENESS AND ETHICAL VARIABLE DESIGN

AI PROMPT USED

"GENERATE PYTHON CODE TO PROCESS EMPLOYEE DETAILS LIKE NAME, GENDER, SALARY."

ORIGINAL AI-GENERATED CODE SNIPPET

```
EMPLOYEES = [
    {"NAME": "JOHN", "GENDER":
"MALE", "SALARY": 50000},
{"NAME": "JANE", "GENDER":
"FEMALE", "SALARY": 45000}
]
 FOR EMP IN
EMPLOYEES:
```

```
    if emp["gender"] == "male":
print(f"Mr. {emp['name']} earns
{emp['salary']}")     else:
print(f"Ms. {emp['name']} earns
{emp['salary']}")
```

Analysis: Non-Inclusive Elements

Uses gender-specific vars/titles (male/female, Mr/Ms). Assumes binary gender, conditions based on gender.

REVISED INCLUSIVE CODE

```
EMPLOYEES = [
    {"NAME": "JOHN", "PRONOUNS": "HE/HIM", "SALARY": 50000},
    {"NAME": "JANE", "PRONOUNS": "SHE/HER", "SALARY": 45000},
{"NAME": "ALEX", "PRONOUNS": "THEY/THEM", "SALARY": 55000}
]
 FOR EMP IN EMPLOYEES:
PRINT(F"{EMP['NAME']} (PRONOUNS: {EMP['PRONOUNS']}) EARNS {EMP['SALARY']}")
```

OUTPUT

```
JOHN (PRONOUNS: HE/HIM)
EARNS 50000
JANE (PRONOUNS: SHE/HER)
EARNS 45000
ALEX (PRONOUNS:
THEY/THEM) EARNS 55000
```

EXPLANATION

CHANGED TO OPTIONAL PRONOUNS, NO GENDER CONDITIONS OR ASSUMPTIONS. PROMOTES INCLUSIVITY FOR NON-BINARY IDENTITIES.